STAFF REVIEW · H446 019 source models

Separate from the eight-page pupil workbook. This copy is in a public repository and is NOT access controlled. Check technical correctness before use.

Printed question mapping: Booklet Q1 = original online Q1; Booklet Q2 = original online Q10

Do Now — source answers

1. An array has a fixed size; a list can change size.

2. The position used to access an array element.

3. There is no next item.

4. At index 1.

I Do — source worked steps

1. Use the start pointer — CODE:
current = 2
data[2] = "Alex"

2. Read next[2] — CODE:
next[2] = 0
current = 0
data[0] = "Ben"

3. Continue pointers — CODE:
next[0] = 3
data[3] = "Cara"

next[3] = 1
data[1] = "Daisy"

4. Reach null — CODE:
next[1] = -1
stop

We Do — guided CHECK

1. Complete the same trace
CODE:
firstIndex = start
secondIndex = next[firstIndex]
secondData = data[secondIndex]

2. Fill the same code
CODE:
int current = ?;
while (current != -1)
{
Console.WriteLine(data[current]);
current = ?;

}
CODE:
int current = start;
while (current != -1)
{
Console.WriteLine(data[current]);
current = next[current];
}

3. Fill the same code
CODE:
next[newIndex] = ?;
start = ?;
CODE:
next[newIndex] = start;
start = newIndex;

4. Fill the same code
CODE:
next[start] = ?;
CODE:
next[start] = next[next[start]];

You Do — source mark schemes

1. Identify the first logical item
CODE:
start = 2
data[start] = "Alex"
start identifies the first logical node. Physical slot 0 holds Ben, but the head can be elsewhere.

2. Follow the next pointers
CODE:
2 → 0 → 3 → 1 → -1
Alex → Ben → Cara → Daisy
The -1 value is a null marker, never a valid array index.

3. Write the traversal loop
CODE:
int current = start;
while (current != -1)
{
Console.WriteLine(data[current]);
current = next[current];
}
The pointer chooses the next logical node; current++ would follow physical slots instead.

4. Count reachable nodes
CODE:
int count=0, current=start;

while(current!=-1) { count++; current=next[current]; }
Console.WriteLine(count); // 4
The arrays have six slots, but only four are in the original chain.

5. Insert after the head
CODE:
data[4] = "Elliot";
next[4] = next[start]; // preserve Ben at index 0
next[start] = 4;
New indexes: 2→4→0→3→1. Names: Alex→Elliot→Ben→Cara→Daisy. Slot 4 must be unused before insertion.

6. Insert a new first node
CODE:
data[4] = "Zoe";
next[4] = start;
start = 4;
On a fresh starter, indexes become 4→2→0→3→1: Zoe→Alex→Ben→Cara→Daisy. Link to the old head before replacing start.

7. Remove the second node
CODE:
if (start != -1 && next[start] != -1)
next[start] = next[next[start]];
On a fresh starter, bypass Ben at index0. Remaining indexes 2→3→1: Alex→Cara→Daisy. The data values are not shifted.

8. Remove the first node safely
CODE:
if (start != -1) start = next[start];
On a fresh starter, new start=0 and the order is Ben→Cara→Daisy. For one node, its next is -1, so start becomes -1. An empty list remains empty without an invalid access.

9. Search by following links
CODE:
int current=start;
while(current!=-1 && data[current]!=name)
current=next[current];
Console.WriteLine(current); // matching index or -1
With a fresh starter: Cara returns 3; an absent name returns -1. Test -1 before accessing data[current]; short-circuit AND prevents invalid indexing.

10. Combine, test and explain
CODE:
Reset before each independent test:
start=-1 → traversal outputs nothing
one reachable node → delete head leaves -1
insert Elliot after Alex → Alex,Elliot,Ben,Cara,Daisy
delete Ben → Alex,Cara,Daisy
no unused slot → reject insertion without changing links

Record the actual results as well as expected results. Reaching an arbitrary predecessor can take O(n); reconnecting a known link is O(1). A removed slot becomes reusable only when the chosen free-slot allocation scheme records it.

Exit Ticket — source answer

1. True. An array-based linked list follows stored links or pointers, not necessarily neighbouring array positions.